Dream Team Flow

Security Deep-Dive

How AI coding agents can ship secure code in regulated industries

The Problem with AI-Generated Code

CodeRabbit production data: AI-authored code introduces 1.75x more logic errors and 2.74x more XSS vulnerabilities

WHAT AI AGENTS GET WRONG

Missing [Authorize] on new endpoints

PII in log statements (SSN, email)

String concatenation in EF queries (SQL injection)

dangerouslySetInnerHTML with user input (XSS)

User A accessing User B's data (broken access control)

Hardcoded secrets in code (API keys, tokens)

CORS set to * on internal APIs

New UserAction without backend enforcement

WHY "BEST EFFORT" ISN'T ENOUGH

CLAUDE.md instructions are advisory

→ Agent can ignore them

Code review is single-pass

→ Reviewer misses what it doesn't look for

No systematic security checklist

→ Random coverage, not guaranteed

No enforcement mechanism

→ Agent can skip any step

Three Layers of Defense

Each layer catches what the others miss — defense in depth

LAYER 1: HOOKS

Can't be bypassed

Migration guard — warns on schema edits

Lock file guard — blocks manual edits

Teammate idle gate — must write notes

Task completed gate — must pass type checks

Tool usage logging — full audit trail

Hooks are shell scripts, not LLM instructions.

An agent cannot argue with a hook.

LAYER 2: OWASP REVIEW

7 categories, every PR

1. SQL injection (EF string concat)
2. Auth/Authz (missing [Authorize])
3. Data exposure (PII in logs, responses)
4. XSS (dangerouslySetInnerHTML)
5. Path traversal (../../etc/passwd)
6. Hardcoded secrets (API keys in code)
7. Insecure defaults (CORS *, no HTTPS)

LAYER 3: DEEP REVIEW

Multi-agent + validation

4 parallel review agents:

- 2x Sonnet (conventions)
- 2x Opus (bugs + security)

Every finding independently verified

Only CONFIRMED issues survive

False positives filtered out

--deep --full: + local builds

DEFENSE IN DEPTH

Hooks prevent. OWASP review detects. Deep review validates. Human review confirms. No single point of failure.

Three-Tier Permission Ladder

From developer sandbox to team-enforced lockdown

LEVEL 1: PERSONAL

Agent works in isolated worktree

Own Docker containers, own ports

No access to production data

Can't push to main

Can't merge PRs

LEVEL 2: SHARED STANDARDS

Conventions enforced via hooks

OWASP scan before PR ready

Type checks must pass

Formatting must be applied

CI must be green (2 round cap)

LEVEL 3: TEAM LOCKDOWN

Schema changes require Mermaid ER diagram + human approval

Domain model changes escalated

PR stays DRAFT until user confirms

Mandatory human review before merge

KEY PRINCIPLE

Agents propose, humans dispose. The system is designed so that AI cannot autonomously change your data model, merge code, or bypass security scans. Every destructive action requires human confirmation.

Audit Trail & Config Health

AUDIT TRAIL

tool-usage-log.sh

Every tool call logged to CSV

Timestamp, session, tool, summary

phase-cost-tracker.sh

Per-phase cost tracking

Compare sessions, spot anomalies

dream-team-history.json

Session metrics per agent

Review rounds, compile rate, tool uses

Git history

Every change committed with ticket ID

Full traceability ticket → PR → code

CONFIG HEALTH SCANNER

config-scan.sh — grades A through F

Checks:

- Secrets in CLAUDE.md / settings.json
- eval in hook scripts (injection vector)
- MCP server count (context overflow risk)
- Token optimization settings
- Memory index size (truncation risk)
- Hook scripts exist and are executable
- Async hooks have timeouts

```
bash ~/.claude/scripts/config-scan.sh
bash ~/.claude/scripts/config-scan.sh --fix
```

The New Threat: Agents as Attack Targets

The attack surface shifted — AI systems are now the target, not just the tool

ATTACK SURFACE EXPANSION

Beyond code quality — AI is the target

Malicious .claude/settings.json in repos → RCE

Malicious skills/plugins with embedded payloads

Agent memory files used for persistence

MCP servers run with full agent permissions

22 prompt injection techniques active in the wild

AI memory poisoning for recommendation manipulation

Opening a repo can now be the attack vector

THE LETHAL TRIFECTA

Simon Willison — all three = critical risk

1. Access to private / sensitive data
2. Reads untrusted content (web, files, repos)
3. Can communicate externally (exfiltrate)

All three present → critical

Any two present → elevated

One alone → manageable

Check: which apply before adding any new capability

REAL CVEs — 2026

Already exploited, not theoretical

CVE-2025-59536 — RCE via .claude/settings.json hooks

Executes before trust dialog, no confirmation

CVE-2025-59536 — API key stolen via ANTHROPIC_BASE_URL

ToxicSkills — 76 malicious in 3,984 skills scanned

13.4% had critical-level security issues

91% combined prompt injection + malicious code

Unit 42 — injection active on real public websites

THE NEW ATTACK MODEL

Attackers don't need to breach your system. They need you to open their repo, install their skill, or visit their page.

Supply Chain Defenses

Malicious repos, skills, and MCP servers — how DTF protects against each

UNTRUSTED REPO HYGIENE

Before running claude . in any repo

Read .claude/settings.json before opening

Check for hooks executing shell commands

Check for enableAllProjectMcpServers: true

Check for ANTHROPIC_BASE_URL overrides

Use --no-mcp flag in unfamiliar repos

Treat repos like code — audit before trust

Never run claude . in a honeypot / untrusted repo

SKILL / MCP HYGIENE

Skills inherit full agent permissions

Only use skills whose source you've read

Read SKILL.md + implementation before installing

Prefer skills you wrote or fully reviewed

MCP: only servers you wrote or vetted completely

Run snyk mcp-scan on existing installs

Check memory files for unauthorized entries

Rotate credentials if a suspect skill ran

DTF SAFEGUARDS IN PLACE

What happens automatically

config-scan.sh: detects eval in hook scripts

config-scan.sh: flags suspicious hook patterns

All marketplace skills are open source + reviewed

No auto-install from untrusted sources

Hook source is always readable — no obfuscation

dtf doctor: validates installation integrity

Worktrees: isolated per ticket, no shared state

SUPPLY CHAIN PRINCIPLE

Install less. Vet everything. Your agent's permissions are your attack surface — same supply chain risk as npm, but with shell access.

Indirect Prompt Injection & Network Containment

When agents read external content — what goes wrong and how DTF limits the blast radius

INDIRECT PROMPT INJECTION

Attacker embeds hidden commands in content

Agent reads content → executes hidden instructions

22 real techniques documented in the wild (Unit 42)

HTML attribute cloaking — 19.8% of attacks

CSS suppression / zero-size elements — 16.9%

Invisible Unicode + homoglyph substitution

Base64-encoded payloads via JavaScript

Social engineering framing — 85.2% of jailbreaks

NETWORK CONTAINMENT

Limit what the agent can reach

GitHub Copilot: agent firewall (allowlist-based)

Claude Code: /sandbox — filesystem + network isolation

DTF worktrees: own containers, own ports

No access to production data or services

curl/wget blocked by default in Claude Code

Web fetch: isolated context window

Agents cannot push to main autonomously — ever

DTF MITIGATION STATUS

What's handled vs what needs vigilance

✓ curl/wget blocked by default

✓ Isolated web fetch context window

✓ No autonomous push/merge, ever

✓ Three-tier permission ladder enforced

✓ PR stays DRAFT until human confirms

⚠ Untrusted files in repos — manual vigilance

⚠ Sandbox mode: available but not default — enable it

THE TRIFECTA CHECK

Before any new agent capability: private data? + reads untrusted content? + can communicate externally? All three = mandatory human gate.

Dream Team Flow

Security is built into the pipeline, not bolted on after

Hooks prevent. OWASP review detects. Deep review validates. Human review confirms.
Full audit trail. Config health scanning. Three-tier permission ladder.